

# **WOMEN'S ENGINEERING COLLEGE**

**Department of Information Science and Engineering**

## **PROJECT REPORT**

**Submitted in partial fulfilment of the requirements**

**for the award of the degree of**

**Bachelor of Technology**

## **FINANCIAL MARKET ANALYSIS DASHBOARD**

**Submitted by**

**N.Punithapriya**

**Register No: 2403121031**

**2024-2028**

| <b>CHAPTER 1</b>                  |                                                         | <b>Pg.No</b> |
|-----------------------------------|---------------------------------------------------------|--------------|
| <b>INTRODUCTION</b>               |                                                         |              |
| <b>1.1</b>                        | <b>Background of Financial Marketing Data Analysis</b>  | <b>1</b>     |
| <b>1.2</b>                        | <b>Motivation and Problem Statement</b>                 | <b>2</b>     |
| <b>1.3</b>                        | <b>Objectives of the Project</b>                        | <b>2</b>     |
| <b>1.4</b>                        | <b>Scope and Limitations</b>                            | <b>3</b>     |
| <b>1.5</b>                        | <b>Organization of the Report</b>                       | <b>3</b>     |
| <br><b>CHAPTER 2</b>              |                                                         |              |
| <b>LITERATURE REVIEW</b>          |                                                         |              |
| <b>2.1</b>                        | <b>Exiting Approaches for Financial Market Analysis</b> | <b>4</b>     |
| <b>2.2</b>                        | <b>Visualization Tools and Dashboards in Finance</b>    | <b>5</b>     |
| <b>2.3</b>                        | <b>Challenges in Financial Market Data Analysis</b>     | <b>5</b>     |
| <b>2.4</b>                        | <b>Gaps This Project Addresses</b>                      | <b>6</b>     |
| <br><b>CHAPTER 3</b>              |                                                         |              |
| <b>SYSTEM ANALYSIS AND DESIGN</b> |                                                         |              |
| <b>3.1</b>                        | <b>Dataset Description</b>                              | <b>7</b>     |
| <b>3.2</b>                        | <b>Source of Financial Data</b>                         | <b>7</b>     |
| <b>3.3</b>                        | <b>Features and Target Variable</b>                     | <b>8</b>     |
| <b>3.4</b>                        | <b>Data Preprocessing</b>                               | <b>8</b>     |
| <b>3.5</b>                        | <b>Feature Engineering</b>                              | <b>9</b>     |
| <b>3.6</b>                        | <b>Tool Selection</b>                                   | <b>9</b>     |
| <b>3.7</b>                        | <b>System Architecture</b>                              | <b>9</b>     |
| <b>3.8</b>                        | <b>Data Pipeline</b>                                    | <b>10</b>    |
| <b>3.9</b>                        | <b>UML Diagram</b>                                      | <b>10</b>    |

## **CHAPTER 4**

### **CONCLUSION AND FUTURE SCOPE**

|            |                                             |    |
|------------|---------------------------------------------|----|
| <b>4.1</b> | Summary of findings                         | 11 |
| <b>4.2</b> | Practical Application in Finance            | 12 |
| <b>4.3</b> | Limitations of Current Work                 | 12 |
| <b>4.4</b> | Future Enhancements and Research Directions | 13 |





# CHAPTER 1

## INTRODUCTION

### 1.1 Background

Financial markets have always been a critical component of the global economy. Stock exchanges around the world process millions of transactions every single trading day, generating vast quantities of data in the process. This data — covering stock prices, trading volumes, sector movements, earnings announcements, and macroeconomic indicators — holds significant information about the health of individual companies and the broader economy. For professionals working in finance, this data is the raw material of every decision they make.

The challenge is that financial data is not self-explanatory. A spreadsheet containing four years of daily closing prices for ten stocks contains real information, but extracting anything meaningful from it requires either significant domain knowledge or the right tools. Professional analysts have both. They work with platforms like Bloomberg Terminal, Refinitiv Eikon, and proprietary brokerage tools that are designed specifically for people who already understand what they are looking at. These platforms are powerful, but they are built for a particular kind of user — experienced, trained, and already comfortable with financial concepts.

For everyone outside that group — finance students, economics undergraduates, retail investors encountering stock data for the first time — the situation is considerably more difficult. The tools available to professionals are either too expensive, too complex, or both. General-purpose tools like Microsoft Excel can handle basic data manipulation but require manual effort to produce useful visualizations and offer no interactivity. Online stock screeners provide some visual output but are designed around trading decisions rather than data exploration and learning.

This creates a real gap. There is a large population of people who want to understand financial market data — not to trade professionally, but to learn how markets behave, how individual stocks move relative to each other, and how specific events affect prices over

time. These users need something different from what currently exists: a tool that prioritizes clarity and accessibility over feature depth, and that allows direct exploration of real market data without requiring a finance degree to operate.

## **1.2 Motivation and Problem Statement**

The motivation for this project came directly from this gap. Finance students frequently encounter stock market data in their coursework but rarely have access to tools that let them explore it interactively. Lecture slides show static charts. Textbook examples use simplified, hypothetical data. The actual Yahoo Finance website provides some visualization but is oriented around current prices and trading activity rather than historical exploration and comparison.

The problem, stated simply, is this: understanding stock market behavior requires direct engagement with real historical data, but the tools that provide access to that data are not designed for learners. Complex interfaces, professional terminology, and feature overload consistently push beginners away before they can extract any understanding from the data they are looking at. What is needed is a dashboard that loads real historical stock data, presents it through clear and readable charts, and allows users to filter and explore without requiring any prior technical or financial knowledge.

## **1.3 Objectives of the Project**

The objectives of this project are as follows:

- To build an interactive dashboard that displays historical stock market data sourced from Yahoo Finance in a clear, readable format accessible to finance students and beginners.
- To implement three visualization types — a line chart for tracking price trends over time, a bar chart for direct stock-to-stock comparison, and a moving average overlay for identifying longer-term patterns beneath short-term price noise.

- To provide functional filtering controls — specifically a stock selector and a date range filter — that allow users to focus on specific stocks and time periods without modifying the underlying dataset.
- To ensure the dashboard loads and responds quickly, targeting a load time of under 3 seconds on a standard connection.
- To build the entire system using open-source Python tools — specifically Python 3.10, Pandas 1.5, Matplotlib 3.7, and Streamlit 1.25 — keeping the project reproducible and accessible without paid software or APIs.

## **1.4 Scope and Limitations**

The scope of this project is deliberately narrow. The dashboard works with a static CSV file downloaded from Yahoo Finance, covering 5 to 10 stocks across the period January 2020 to December 2023. This four-year window was chosen because it captures several distinct market phases — the COVID-19 crash of March 2020, the sharp recovery through 2021, and the correction that followed Federal Reserve rate increases in 2022 — making it useful for a range of analytical exercises.

The project does not attempt price prediction, forecasting, or investment advice of any kind. It does not connect to a live data feed. The data ends in December 2023. The moving average is the only analytical layer applied. No statistical models, machine learning algorithms, or predictive features are included in this version of the project.

## **1.5 Organization of the Report**

Chapter 2 reviews existing work in financial data analysis and visualization, identifying the specific gap that this project addresses. Chapter 3 covers system analysis and design — the dataset, preprocessing steps, tool selection, system architecture, and data pipeline. Chapter 6 reflects on what the project achieved, where it falls short, and what the logical next steps would be for future development.

## **CHAPTER 2**

### **LITERATURE REVIEW**

#### **2.1 Existing Approaches to Financial Market Analysis**

The academic literature on financial market analysis is extensive, and the majority of it is focused on prediction. The central question driving most research in this space is whether future stock prices can be reliably forecast from historical data, and if so, what methods produce the most accurate results.

Early approaches relied heavily on statistical techniques. Time series models such as ARIMA (AutoRegressive Integrated Moving Average) were widely used through the 1980s and 1990s to model sequential price data and produce short-term forecasts. These methods work reasonably well under stable market conditions but struggle to adapt when market behavior changes abruptly — as it does during crashes, policy shifts, or major geopolitical events.

The shift toward machine learning beginning in the early 2000s brought more flexible approaches. Random Forest became popular in financial applications because of its resistance to overfitting and its ability to handle high-dimensional, noisy datasets. Support Vector Machines were applied to directional prediction — classifying whether a stock's price would rise or fall over a given period. Deep learning approaches, particularly Long Short-Term Memory networks, have received significant research attention for short-term price forecasting.

This project uses none of these methods. The reason is straightforward: prediction is not the goal. The dashboard is built for exploration and understanding, not for generating forecasts. Reviewing the prediction literature is useful context — it shows what the field has prioritized and why a visualization-first approach fills a genuinely different need.

## 2.2 Visualization Tools and Dashboards in Finance

Research on financial visualization tools is considerably less extensive than research on prediction, which itself reflects a gap in the field. Most tools designed for financial data analysis assume the user already understands what they are looking at and wants to act on it.

Commercial platforms like Bloomberg Terminal and Refinitiv Eikon are the dominant professional tools. They provide comprehensive charting capabilities, real-time data feeds, news integration, and analytical tools built for institutional users. The cost of a Bloomberg Terminal subscription places these tools entirely out of reach for students and individual learners.

Academic tools built on Python have grown in relevance as data science education has expanded. Streamlit has emerged as a practical framework for building lightweight data dashboards that can be shared and run without web development expertise. Its Python-native interface, fast prototyping workflow, and built-in support for interactive widgets make it a practical choice for projects where the priority is getting meaningful output in front of users quickly.

## 2.3 Challenges in Financial Market Data Analysis

Working with financial data presents several practical challenges that affect any project in this domain, regardless of whether the goal is prediction or visualization.

Data quality is the first challenge. Stock market data sourced from public APIs and CSV exports frequently contains missing values — gaps caused by public holidays, trading halts, or simply missing records in the source database. These gaps need to be handled carefully. Dropping rows introduces discontinuities in time series charts that can mislead users. Forward filling preserves continuity but introduces a minor distortion at the gap point.

Data volume is the second challenge. A daily price dataset covering ten stocks across four years contains approximately 10,000 rows. While this is not large by modern data processing standards, rendering it interactively in a web-based dashboard requires careful management of when filtering happens relative to when rendering happens.

Volatility and outliers present a third challenge. Financial data contains genuine extreme values — the March 2020 crash saw the S&P 500 drop approximately 34% in five weeks. These are not errors; they are real events that carry significant information. Any preprocessing step that treats them as noise risks distorting the historical record.

## **2.4 Gap This Project Addresses**

The gap this project addresses is specific. Existing financial analysis tools fall into two categories: professional-grade platforms built for experienced users, and prediction-focused research tools built for technical audiences. Neither category serves the user who wants to develop financial literacy through direct exploration of real market data.

What is missing is a simple, open-source, locally runnable dashboard that loads real historical data, presents it clearly, and allows filtering by stock and date without requiring the user to understand anything beyond basic stock market concepts. This project builds that tool.

## **CHAPTER 3**

### **SYSTEM ANALYSIS AND DESIGN**

#### **3.1 Dataset Description**

The dataset used in this project consists of historical daily stock market data for 5 to 10 publicly listed companies, downloaded from Yahoo Finance in CSV format. The data covers the period from January 2020 to December 2023, a four-year window selected because it encompasses several distinct and well-documented market phases.

The period begins just before the COVID-19 pandemic reached global scale. The market crash of February and March 2020, during which major indices lost between 30 and 35 percent of their value in approximately five weeks, is captured in full. The subsequent recovery through 2020 and 2021, and the 2022 correction triggered by Federal Reserve interest rate increases, are also present. Total row count falls between approximately 1,000 and 3,000 rows depending on how many stocks are loaded. Each row represents one trading day.

#### **3.2 Source of Financial Data**

All data used in this project was downloaded from Yahoo Finance. Yahoo Finance provides free historical OHLCV data — open, high, low, close, and volume — for publicly listed stocks across major global exchanges. Yahoo Finance was selected for three reasons: the data is publicly available at no cost, the CSV export format is consistently structured across different stocks and date ranges, and the historical data for major stocks is complete and accurate for visualization purposes.

No API connection is used in this project. Data is loaded from a locally stored CSV file. This decision was made to simplify the development environment and avoid dependency on API rate limits or authentication during development in Google Colab.

### 3.3 Features and Target Variables

Each row in the dataset contains the following six fields:

- Date: The calendar date of the trading day.
- Open: The price of the stock at market open.
- High: The highest price reached during the trading day.
- Low: The lowest price reached during the trading day.
- Close: The price at market close. This is the primary field used across all three chart types.
- Volume: The total number of shares traded during the day.

No target variable is defined. This project visualizes historical behavior rather than predicting future prices. Closing price is the primary measure of stock performance across all visualizations.

### 3.4 Data Preprocessing

Two preprocessing steps are applied to the raw CSV data before it reaches the dashboard interface. Both steps run once at application startup, before any user interaction takes place.

#### 3.4.1 Handling Missing Values

Yahoo Finance CSV exports occasionally contain missing values, typically around public holidays or trading halts. Missing values are handled using forward fill — the last valid value is carried forward to fill the gap. This approach was chosen over row deletion because dropping rows would create discontinuities in the line chart that could be misread as sharp price movements.

#### 3.4.2 Normalization and Date Parsing

The date column in the raw CSV is stored as a string and is converted to a Pandas datetime object using `pd.to_datetime()` during preprocessing. Without this step, date-based filtering in the Streamlit interface does not function correctly.

Normalization is deliberately not applied. The dashboard displays raw closing prices and volume figures. Scaling the values would remove the actual price context that users need to read the charts meaningfully.

### **3.5 Feature Engineering**

The only derived feature is the moving average — a rolling mean of the closing price calculated over a fixed window of trading days, computed using Pandas' `rolling().mean()` method. A window of 20 days approximates one calendar month of trading activity. A window of 50 days provides a longer-term view and is less sensitive to short-term fluctuations. The moving average column is calculated once at load time and stored in the dataframe. It smooths daily price noise and makes the underlying trend visible in the line chart.

### **3.6 Tool Selection**

This project does not use a machine learning model. The tool choices reflect the visualization-first approach:

- Python 3.10: Runtime environment, developed and tested in Google Colab.
- Pandas 1.5: Data loading, cleaning, filtering, and moving average calculation.
- Matplotlib 3.7: Rendering the three chart types — line, bar, and moving average overlay.
- Streamlit 1.25: Dashboard interface, stock selector dropdown, and date range filter.

### **3.7 System Architecture**

The system is organized into three distinct layers, each with a clearly defined responsibility.

**Data Layer:** Loads the Yahoo Finance CSV into a Pandas dataframe at startup. All preprocessing — forward fill, date parsing, moving average calculation — runs once here, before any user interaction.

Logic Layer: Handles filtering. When the user selects a stock or adjusts the date range, Pandas filters the in-memory dataframe. The CSV is not re-read on each interaction.

Presentation Layer: Takes the filtered dataframe and passes it to Matplotlib for rendering. Streamlit wraps the output and handles the controls — stock selector dropdown, date range inputs, and chart display.

### **3.8 Data Pipeline**

The complete data pipeline flows as follows:

1. CSV file loaded from local storage into a Pandas dataframe using `pd.read_csv()`.
2. Date column parsed from string format to datetime using `pd.to_datetime()`.
3. Missing values filled using Pandas forward fill method.
4. Moving average columns calculated and appended to the dataframe.
5. User selects stock and date range via Streamlit controls.
6. Dataframe filtered in memory based on the user selection.
7. Filtered data passed to Matplotlib for chart rendering.
8. Charts displayed in the Streamlit dashboard interface.

### **3.9 UML Diagrams**

Use Case Diagram: The primary actor is the user. Use cases include: select stock, set date range, view line chart, view bar chart, and view moving average chart. No authentication or admin role exists — the dashboard is single-user by design.

Activity Diagram: User opens dashboard — system loads CSV and preprocessing runs — user selects stock and date range — system filters dataframe — charts render — user views output. If the user changes the selection, the filter and render steps repeat.

Component Diagram: Three components — Data (CSV file), Processing (Pandas), and Presentation (Matplotlib and Streamlit). Data flows one way: CSV to Pandas to Matplotlib to Streamlit interface. There is no feedback loop.

## **CHAPTER 4**

### **CONCLUSION AND FUTURE SCOPE**

#### **4.1 Summary of Findings**

This project set out to solve a specific and practical problem: financial market data is publicly available, historically rich, and genuinely useful for learning — but the tools that provide access to it are not built for learners. The Financial Market Analysis Dashboard addresses that problem by loading real historical stock data from Yahoo Finance and presenting it through a clean, interactive interface that a finance student can use without prior technical or financial expertise.

The dataset covers three stocks — AAPL (Apple Inc.), TSLA (Tesla Inc.), and TCS (Tata Consultancy Services) — across January 2020 to December 2023. These three stocks represent different sectors, geographies, and growth profiles, providing a dataset that is genuinely varied and useful for demonstrating comparison and trend visualization capabilities.

The most significant finding concerns the March 2020 market crash. All three stocks dropped sharply within a three to four week window. The drop is visible immediately on the line chart without annotation. The recovery trajectories diverged: TSLA showed the highest growth across the full four-year period, rising by approximately 700 percent from its March 2020 low to its late 2021 peak. TCS followed a quieter, more consistent trajectory. AAPL recovered strongly through 2021 before consolidating through 2022 as rising interest rates compressed technology sector valuations.

The moving average overlay added genuine analytical value. The 20-day moving average applied to TSLA made the upward trend through 2021 and the reversal through 2022 clearly legible, smoothing the daily noise that makes the raw price line difficult to read during volatile periods.

Classmate testing confirmed the interface achieved its primary goal. Users navigated the stock selector and date filter without instruction. Load time ran consistently at 2 to 3

seconds. The most consistent feedback was a request for a zoom feature on the line chart and a slight slowdown when all three stocks were loaded simultaneously on lower-end devices.

## **4.2 Practical Applications in Finance**

The dashboard has concrete utility in three settings.

**Undergraduate Finance Education:** A course covering equity markets, market cycles, or the impact of macroeconomic events on stock prices can use this dashboard as a live demonstration tool. The date filter allows a lecturer to isolate any specific market event — the March 2020 crash, the 2021 technology rally, the 2022 rate-hike correction — and show students how individual stocks responded in real time.

**Independent Study and Financial Literacy:** Students developing an interest in equity markets outside coursework need a low-barrier entry point. This dashboard provides one — no installation beyond a Python environment, no API key, no account creation. A student who downloads the CSV and runs the Streamlit application can be exploring four years of real market data within minutes.

**Foundation for Further Development:** The dashboard serves as a working reference implementation for students building their own data analysis tools. The preprocessing pipeline covers the most common steps in a financial data workflow. Students extending this project have a clean, documented starting point rather than building from scratch.

## **4.3 Limitations of Current Work**

Four limitations are worth stating directly.

**Static Dataset with Fixed End Date:** The dashboard works with a CSV file ending in December 2023. Anyone using the dashboard in 2025 is looking at data that is over a year old and does not reflect current market conditions. The static CSV approach was the right choice for development but creates a real constraint on practical usefulness.

**No Zoom or Interactive Navigation:** The line chart for TSLA across the full four-year period is visually compressed. A user who wants to examine a specific quarter cannot zoom in — the only option is to use the date filter. Zoom was the most consistent piece of feedback from classmate testing.

**Performance with Multiple Stocks:** Loading all three stocks simultaneously caused a measurable slowdown on lower-end devices. The bar chart in particular took noticeably longer to redraw. This is a user experience problem that would worsen if the dataset were expanded.

**No Volume Visualization:** The Yahoo Finance CSV includes a daily trading volume column. Volume is a standard secondary indicator in technical analysis. The dashboard does not visualize it, which means the dashboard presents an incomplete picture of what the data actually contains.

#### **4.4 Future Enhancements and Research Directions**

The most valuable future enhancements follow directly from the limitations above, listed in order of impact relative to implementation effort.

**Live Data Feed via yfinance API:** Replacing the static CSV with a live data feed using the yfinance Python library is the single highest-impact improvement. The `yf.download()` function fetches current historical data without requiring an API key. Replacing `pd.read_csv()` with `yf.download()` would keep the dashboard current automatically. The rest of the architecture requires no changes.

**Interactive Charts Using Plotly:** Replacing Matplotlib with Plotly would add zoom, pan, hover tooltips, and interactive legend controls to all three chart types. Plotly integrates with Streamlit directly through `st.plotly_chart()`. Zoom was the most requested feature in user testing and Plotly provides it natively.

**Volume Overlay:** Adding a volume bar chart below the price line chart would give users a second dimension of information already present in the dataset. This is the lowest-effort enhancement with meaningful analytical value — one additional Matplotlib subplot using a column already in the dataframe.

**Expanded Stock Coverage:** Expanding from three stocks to ten or more — across multiple sectors and geographies — would make the comparison features substantially more useful. The preprocessing pipeline handles any number of stocks without modification. The performance issue would need to be addressed first.

**Stock Price Prediction Using Machine Learning:** Adding an LSTM-based forecasting layer is the most substantial long-term enhancement. This feature was planned but not implemented due to time constraints. Implementing it correctly — with proper train-test splitting, validation, and clear communication that forecasts are probabilistic — represents a significant additional project. It is the right direction for a future version, but the live data feed and interactive charting improvements should come first.

**Interface Improvements:** Minor improvements identified during testing include a monthly filter option, a chart type toggle, and a summary statistics panel showing key figures — highest price, lowest price, and percentage change — for the selected stock and date range. These are Streamlit widget additions that require no architectural changes.